



Modularización y Buenas Prácticas en Kedro

Separación en paquetes y módulos (`src/` en Kedro)

La estructura de directorios en Kedro sigue un enfoque modular y bien organizado que facilita la escalabilidad y el mantenimiento de proyectos de ciencia de datos (Kedro, s.f.).

El directorio `src/` es el núcleo de un proyecto Kedro y contiene todo el código fuente organizado de manera sistemática. Este enfoque de organización sigue los principios de arquitectura modular, permitiendo que proyectos complejos mantengan una estructura coherente incluso al crecer.

La estructura básica del directorio es la siguiente:

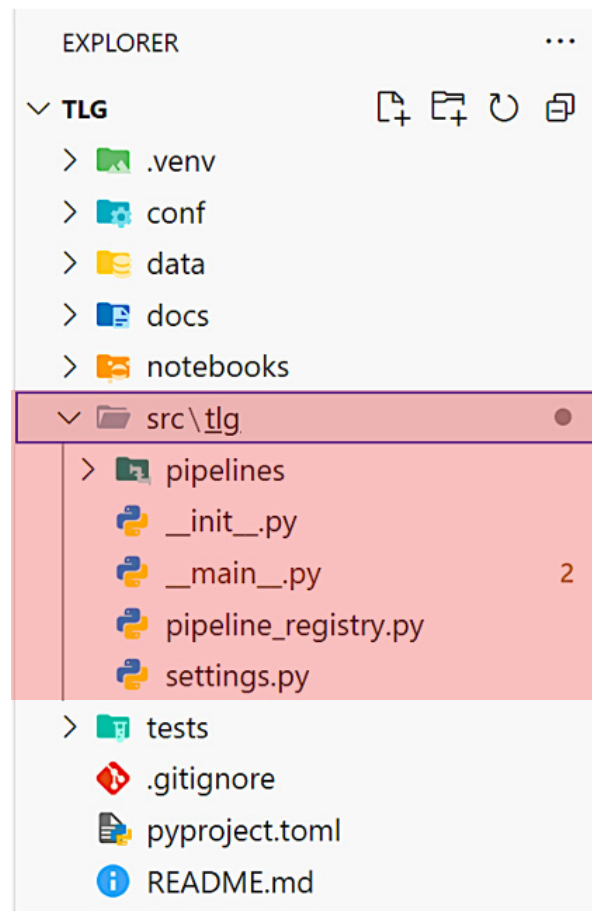


Figura 1. Estructura básica de un directorio en Kedro.

Propósito y fundamentos

El directorio `src/` implementa el patrón de paquete de Python, lo que permite:

1. Facilitar las referencias entre módulos usando rutas de importación claras y consistentes.
2. Instalar el proyecto como un paquete Python, facilitando la distribución y despliegue.
3. Separar código fuente y archivos de configuración, datos o documentación.
4. Crear una estructura de archivos que permite la creación de pruebas unitarias e integración.

Principios de organización modular

La organización modular promueve:

- **Alta cohesión:** cada módulo presenta responsabilidad clara y bien definida.
- **Bajo acoplamiento:** los módulos minimizan las dependencias entre sí.
- **Reutilización:** el código se organiza para facilitar su reutilización en diferentes contextos.
- **Mantenibilidad:** la estructura es fácil de entender y mantener.

Aplicación de SOLID en Kedro

Los principios SOLID de la programación orientada a objetos se aplican en Kedro de la siguiente manera:

- **Principio de responsabilidad única (SRP):** cada nodo realiza una única función específica. Por ejemplo, un nodo para limpieza de datos no debe realizar también el entrenamiento de modelos.
- **Principio abierto/cerrado (OCP):** los *pipelines* están diseñados para ser extensibles sin modificar el código existente, mediante la adición de nuevos nodos.
- **Principio de sustitución de Liskov (LSP):** los *datasets* en el catálogo pueden ser reemplazados por otros del mismo tipo (por ejemplo, cambiar un CSV por un Parquet) sin afectar la funcionalidad.
- **Principio de segregación de interfaces (ISP):** los nodos tienen interfaces mínimas, definidas por sus entradas y salidas específicas.
- **Principio de inversión de dependencias (DIP):** los *pipelines* dependen de abstracciones (nodos) en lugar de implementaciones concretas, facilitando el reemplazo y prueba.

Anatomía del directorio `src/`

El paquete principal, **cuyo nombre coincide con el nombre del proyecto**, contiene toda la lógica específica del proyecto:

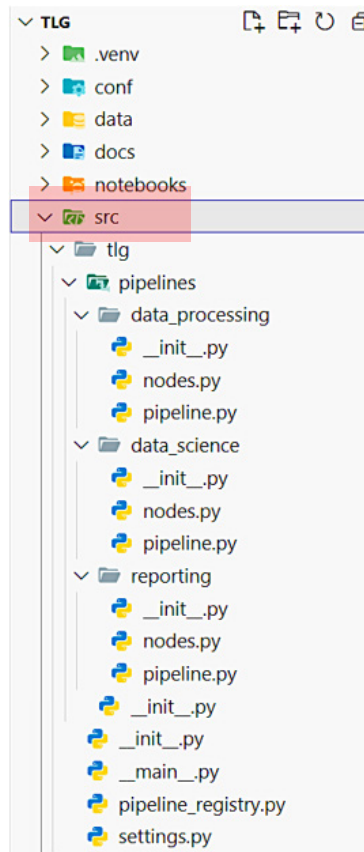


Figura 2. Anatomía de un directorio `src/`.

Esta estructura fue creada por **kedro new** con la opción de agregar un *pipeline* de ejemplo.

- **`__init__.py`**: archivo de Python que indica que el directorio es un paquete de Python.
- **`pipeline_registry.py`**: contiene código para la orquestación de todos los *pipelines*.
- **`settings.py`**: configuración global del proyecto.

El directorio **pipelines** contiene los diferentes *pipelines* del proyecto, cada uno organizado como un módulo independiente:

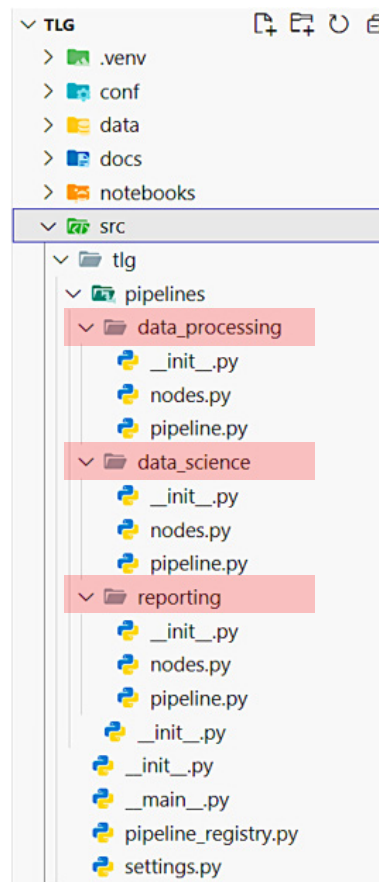


Figura 3. Directorio de *pipelines*

Cuando se crea un proyecto agregando un ejemplo, se agregan tres *pipelines* diferentes: **data_processing**, **data_science** y **reporting**.

Cada subdirectorio de un *pipeline* contiene:

- **nodes.py**: funciones individuales que realizan tareas específicas.
- **pipeline.py**: define el *pipeline* conectando los nodos en un flujo de trabajo.
- **__init__.py**: hace que el directorio sea un paquete Python y expone las funciones necesarias.

Mejores prácticas para la separación en módulos

Organización por dominio o propósito

Organiza los *pipelines* según su propósito o dominio de negocio.

Ejemplos comunes:

- **data_engineering**: para la ingesta, limpieza y transformación de datos.
- **data_science**: para el modelado estadístico y aprendizaje automático.
- **feature_engineering**: para la creación y selección de características.
- **reporting**: para la generación de visualizaciones e informes.
- **monitoring**: para métricas de monitoreo de modelos en producción.
- **deployment**: para preparación de modelos para despliegue.

Algunos ejemplos aplicados serían:

Para proyectos de sistemas de recomendación:

```
pipelines/  
├─ data_collection/      # Recolección y unificación de datos  
├─ user_profiling/      # Creación de perfiles de usuario  
├─ item_processing/     # Procesamiento de características de ítems  
├─ recommender/         # Algoritmos de recomendación  
└─ feedback_analysis/   # Análisis de feedback y reentrenamiento
```

Para proyectos de detección de fraude:

```
pipelines/  
├─ transaction_processing/ # Procesamiento de transacciones  
├─ feature_extraction/     # Extracción de características  
├─ anomaly_detection/      # Detección de anomalías  
├─ rule_engine/           # Motor de reglas para clasificación  
└─ alert_generation/      # Generación y priorización de alertas
```

Figura 4. Ejemplo de separación de módulos.

Recuerda que puedes crear nuevos *pipelines* con la instrucción:

```
kedro pipeline create <nombre_pipeline>
```

Y este comando generará automáticamente la estructura de carpetas necesaria.

■ Granularidad adecuada de los nodos

Cada nodo debe realizar una única tarea bien definida, siguiendo el principio de responsabilidad única:

```
# Ejemplo de buena granularidad en nodes.py

def clean_missing_values(data: pd.DataFrame) -> pd.DataFrame:
    """Limpia valores faltantes en el dataset."""
    return data.fillna(0)

def normalize_features(data: pd.DataFrame) -> pd.DataFrame:
    """Normaliza las características numéricas."""
    numeric_columns = data.select_dtypes(include=['float64',
                                                    'int64']).columns

    data[numeric_columns] = (
        data[numeric_columns] - data[numeric_columns].mean()
    ) / data[numeric_columns].std()

    return data
```

Figura 5. Principio de responsabilidad única.

Criterios para determinar la granularidad óptima:

1. **Complejidad cognitiva:** si una función requiere más de 10-15 líneas o tiene múltiples bloques anidados, es posible que pueda dividirse.
2. **Reusabilidad:** si parte de la lógica puede reutilizarse en otro contexto, hay que separarla en otro módulo.
3. **Testabilidad:** cada nodo debe poder probarse de forma aislada.
4. **Parametrización:** si una función tiene muchos parámetros o configuraciones, es posible dividirla.
5. **Tiempo de ejecución:** nodos que toman mucho tiempo de ejecución pueden subdividirse para facilitar puntos de prueba y recuperación.

■ Reutilización

Kedro no permite la duplicación directa de *pipelines* porque todos los nodos dentro de un proyecto deben tener nombres únicos, sin embargo, el uso de espacios de nombres (*namespaces*) ayuda a resolver este problema al aislar *pipelines* idénticos, además de mejorar la visualización en Kedro-Viz.

```
def create_pipeline(**kwargs) -> Pipeline:
    return pipeline(
        pipeline_existente, # pipeline existente
        inputs={"old_input_name": "new_input_name"}, # Mapear entrada en pipeline existente a nueva
        entrada
        outputs={"old_output_name": "new_output_df_name"}, # Mapear salida en pipeline existente a
        nueva salida
        parameters={"params:model_options": "params:new_model_options"}, # Actualizar parámetros
    )
```

Figura 6. Ejemplo de reutilización.

Para crear un nuevo *pipeline* que realiza la misma tarea, pero con diferentes entradas, salidas y parámetros, se puede utilizar la misma función de creación `pipeline()` indicando el nombre del *pipeline* existente como primer parámetro.

Ahora bien, dado que Kedro no permite tener *pipelines* con los mismos nombres de nodo en la ejecución, para facilitar la reutilización de código se ofrece la **funcionalidad de namespaces**.

Un *namespace* es una forma de aislar los nodos, entradas, salidas y parámetros dentro de un *pipeline*. Si se añade el atributo `namespace="nombre_del_namespace"` dentro de la función `pipeline()`, se agregará el prefijo `nombre_del_namespace` a todos los nodos, entradas, salidas y parámetros del nuevo *pipeline*.

Para conservar el nombre de las entradas, salidas y parámetros, hay que listarlos como se muestra a continuación:

```
pipeline(  
    [node(...), node(...), node(...)],  
    namespace="namespace_name",  
    inputs={"first_input_to_not_be_prefixed", "second_input_to_not_be_prefixed"},  
    outputs={"first_output_to_not_be_prefixed", "second_output_to_not_be_prefixed"},  
    parameters={"first_parameter_to_not_be_prefixed",  
                "second_parameter_to_not_be_prefixed"},  
)
```

Figura 7. Ejemplo para conservar el nombre de las entradas, salidas y parámetros usando *namespaces*.

■ Organización de código adicional

Para código auxiliar que no forma parte directa de los *pipelines*, se pueden crear módulos adicionales. Es importante mantener la estructura estándar de un paquete en Python.

```
src/
├── <package_name>/
│   ├── __init__.py
│   ├── pipeline_registry.py
│   ├── settings.py
│   ├── pipelines/
│   │   └── ...
│   └── extras/          # Código adicional
│       ├── __init__.py
│       ├── utils.py      # Utilidades generales
│       ├── visualization.py # Funciones de visualización
│       └── metrics.py    # Métricas personalizadas
│   └── io/              # Extensiones de E/S personalizadas
│       ├── __init__.py
│       └── custom_datasets.py
```

Figura 8. Ejemplo de módulos adicionales con estructura de Python.

■ Descubrimiento automático de *pipelines*

Desde Kedro 0.18.3, la función `find_pipelines()` permite el descubrimiento automático de *pipelines*, para ello:

1. Busca en el directorio `src/<package_name>/pipelines/`
2. Importa cada módulo `<package_name>.pipelines.<pipeline_name>`
3. Llama a la función `create_pipeline()` de cada módulo.
4. Valida que el objeto construido sea un *pipeline*.

Para que un *pipeline* sea autodescubierto, debe:

- Estar en el directorio correcto (`pipelines/<pipeline_name>/`).
- Tener una función `create_pipeline()` que devuelva un objeto *pipeline*.
- Ser importable (tener los archivos `__init__.py` necesarios).

Uso de `hooks.py` para personalizar la ejecución del *pipeline*

Los *hooks* son funciones que se ejecutan en puntos predefinidos (eventos) durante la ejecución de un *pipeline* en Kedro. Permiten:

- Agregar funcionalidad personalizada.
- Alterar comportamientos existentes.
- Responder a eventos durante la ejecución.
- Integrar Kedro con servicios externos.

Kedro ofrece varios tipos de *hooks* que corresponden a diferentes momentos en el ciclo de vida de ejecución:

Hooks de pipeline

Nombre	Descripción	Uso típico
<code>before_pipeline_run</code>	Antes de ejecutar el <i>pipeline</i>	Configuración, notificaciones, registro inicial
<code>after_pipeline_run</code>	Después de ejecutar el <i>pipeline</i>	Limpieza, notificaciones, métricas finales
<code>on_pipeline_error</code>	Cuando ocurre un error durante la ejecución	Manejo de errores, notificaciones de fallos

Hooks de nodos

Nombre	Descripción	Uso típico
<code>before_node_run</code>	Antes de ejecutar un nodo	<i>Logging</i> , validación de entradas, medición de tiempo
<code>after_node_run</code>	Después de ejecutar un nodo	Métricas, validación de resultados, <i>caching</i>
<code>on_node_error</code>	Cuando ocurre un error en un nodo	Manejo de errores específicos, reintentos

Hooks de catálogo

Nombre	Descripción	Uso típico
<code>before_dataset_loaded</code>	Antes de cargar un <i>dataset</i>	Validación, <i>logging</i> , métricas
<code>after_dataset_loaded</code>	Después de cargar un <i>dataset</i>	Transformación, validación, estadísticas
<code>before_dataset_saved</code>	Antes de guardar un <i>dataset</i>	Transformación, validación
<code>after_dataset_saved</code>	Después de guardar un <i>dataset</i>	Notificaciones, métricas de almacenamiento

Durante una ejecución de `kedro after_dataset_saved run`, los *hooks* se ejecutan en el siguiente orden:

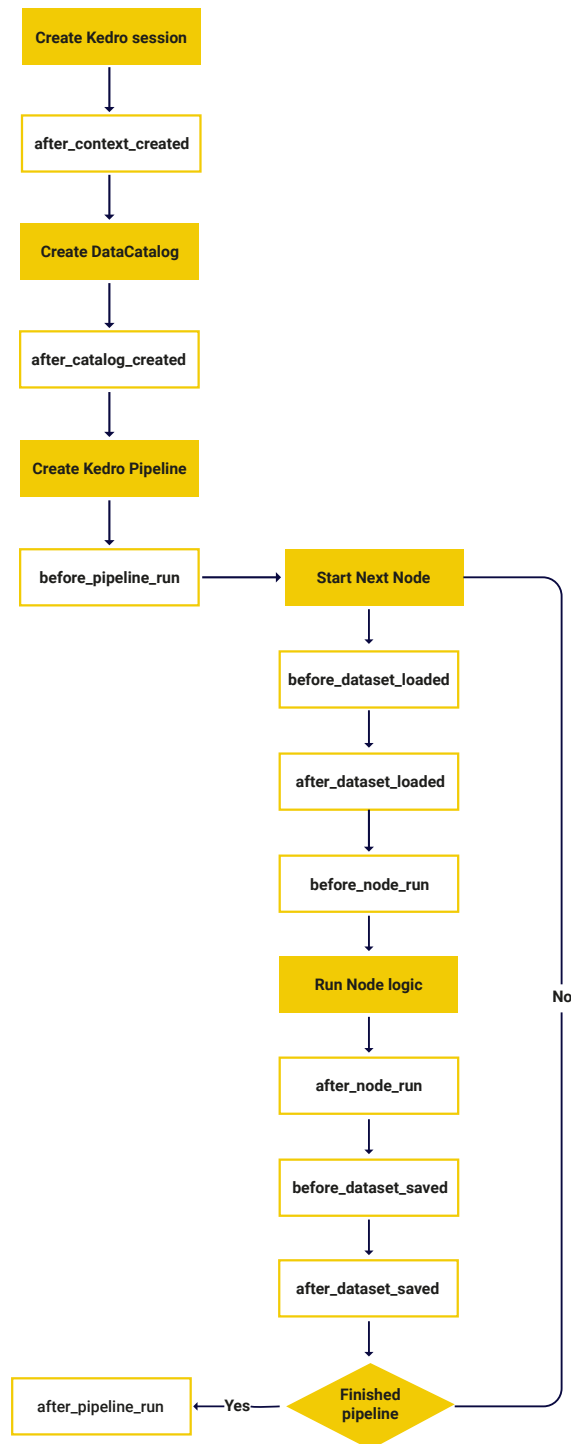


Figura 9. Orden de ejecución de los *hooks*.

Implementación de *hooks*

Creación de una clase de *hooks*

1. Crear o modificar el archivo `src/<paquete>/hooks.py` para definir la implementación del *hook* dentro de una clase.
2. Registrar esa clase en `src/<paquete>/settings.py` en la variable **HOOKS**.
3. La implementación del *hook* se indica con el decorador `@hook_impl` y el nombre del *hook* para el método dentro de la clase.

```
# src/<package_name>/hooks.py
import logging

from kedro.framework.hooks import hook_impl
from kedro.io import DataCatalog

class DataCatalogHooks:
    @property
    def _logger(self):
        return logging.getLogger(__name__)

@hook_impl
def after_catalog_created(self, catalog: DataCatalog) -> None:
    self._logger.info(catalog.list())
```

Por ejemplo, la definición de un *hook* que se ejecutará después de la creación del catálogo sería como sigue:

```
# src/<package_name>/settings.py
from <package_name>.hooks import DataCatalogHooks

HOOKS = (DataCatalogHooks())
```

Figura 10. Ejemplo de definición de un *hook*.

Después de la definición del *hook*, se tiene que registrar en `settings.py`. Se puede registrar más de una implementación para un *hook*, y se ejecutarán en orden “Últimas Entradas, Primeras Salidas” (LIFO – *Last In, First Out*).

Algunos ejemplos de *hooks*:

Hook para validar los datos antes de la ejecución de los nodos y después de la carga del catálogo.

```
# src/<package_name>/hooks.py
class DataValidationHooks:
    """Hooks para validar datos antes y después de operaciones."""

    @hook_impl
    def before_node_run(self, node: Node, inputs: Dict[str, Any]) -> None:
        """Validar entradas del nodo."""
        for name, data in inputs.items():
            if hasattr(data, "empty") and data.empty:
                raise ValueError(f"El dataset '{name}' está vacío")

    @hook_impl
    def after_dataset_loaded(self, dataset_name: str, data: Any) -> None:
        """Validar datos después de cargarlos."""
        if hasattr(data, "isnull") and data.isnull().sum().sum() > 0:
            logging.warning(f"El dataset '{dataset_name}' contiene valores nulos")
```

Figura 11. Ejemplo de *hook* para validar datos.

Hook para enviar notificaciones por correo electrónico al terminar la ejecución del *pipeline* o ante algún error.

```
# src/<package_name>/hooks.py
import smtplib
from email.message import EmailMessage

class EmailNotificationHooks:
    """Hooks para enviar notificaciones por correo electrónico."""

    def __init__(self, recipients):
        self.recipients = recipients

    @hook_impl
    def after_pipeline_run(self) -> None:
        """Enviar notificación de éxito."""
        self._send_email(
            subject="Pipeline completado con éxito",
            content="El pipeline se ha ejecutado correctamente.",
        )

    @hook_impl
    def on_pipeline_error(self, error, run_params) -> None:
        """Enviar notificación de error."""
        self._send_email(
            subject="Error en el pipeline",
            content=f"El pipeline ha fallado con el error: {str(error)}",
        )

    def _send_email(self, subject, content):
        # Lógica para enviar correos electrónicos
        # (Configuración de SMTP, credenciales, etc.)
        pass
```

Figura 12. Ejemplo de *hook* para enviar notificaciones por correo electrónico.

En entornos de producción, los *hooks* son particularmente útiles para:

- Monitoreo y alertas
- Registro para auditoría

Los *hooks* en Kedro ofrecen una forma flexible y poderosa de personalizar la ejecución de *pipelines* sin modificar el código base.

Son ideales para:

- Extender la funcionalidad de Kedro sin modificar su código central.
- Integrar Kedro con sistemas externos como herramientas de MLOps o monitoreo.
- Implementar validaciones y verificaciones en diferentes etapas del *pipeline*.
- Agregar *logging* especializado para depuración y auditoría.
- Optimizar el rendimiento con estrategias como *caching*.

Al utilizar *hooks* de manera efectiva, los equipos pueden adaptar Kedro a sus necesidades específicas, creando flujos de trabajo más robustos, monitoreables y mantenibles.

Uso de bitácoras

Las bitácoras (*logs*) son una parte crucial de cualquier aplicación de datos, especialmente en entornos de producción. **Kedro ofrece un sistema de *logging* flexible que permite monitorear, depurar y auditar la ejecución de *pipelines*.**

Kedro utiliza el módulo de *logging* estándar de Python, al que podrás acceder a través del enlace que te dejamos en la sección de Material adicional, pero lo extiende con configuraciones predeterminadas y opciones adicionales. Por defecto, Kedro configura el *logging* para mostrar mensajes en la consola y guardarlos en archivos.

La configuración define:

- Los ***handlers*** (manejadores) que determinan a dónde se envían los *logs*.
- Los ***formatters*** (formateadores) que definen cómo se formatean los mensajes.
- Los ***loggers*** (objetos de bitácora) que determinan qué mensajes se capturan.
- El nivel de detalle (***level***) para cada *logger*.



La configuración por omisión de Kedro está definida como se ve a continuación. Para acceder al código, te compartimos un enlace en la sección de Material adicional.

```
version: 1
disable_existing_loggers: False

formatters:
  simple:
    format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"

handlers:
  console:
    class: logging.StreamHandler
    level: INFO
    formatter: simple
    stream: ext://sys.stdout

  info_file_handler:
    class: logging.handlers.RotatingFileHandler
    level: INFO
    formatter: simple
    filename: info.log
    maxBytes: 10485760 # 10MB
    backupCount: 20
    encoding: utf8
    delay: True

loggers:
  kedro:
    level: INFO
```

Figura 13. Configuración de Kedro por omisión.

En esta configuración vemos que el nivel de bitácora es **INFO** y envía todos los registros aun archivo llamada **info.log**.

Los **niveles de logging** se utilizan para categorizar la importancia o severidad de los mensajes que se registran durante la ejecución de un programa.

El módulo estándar **logging** define los siguientes niveles, en orden ascendente de severidad:

Nivel	Función	Uso recomendado
DEBUG	<code>logger.debug()</code>	Información detallada para depuración durante desarrollo
INFO	<code>logger.info()</code>	Eventos normales del proceso, útiles para seguimiento
WARNING	<code>logger.warning()</code>	Situaciones inesperadas, pero no críticas y que no detienen la ejecución del programa
ERROR	<code>logger.error()</code>	Errores que impiden realizar una operación específica
CRITICAL	<code>logger.critical()</code>	Errores graves que podrían causar que toda la aplicación falle

Se puede personalizar la configuración básica con un archivo YAML, para eso se recomienda que se use: `conf/base/logging.yml`.

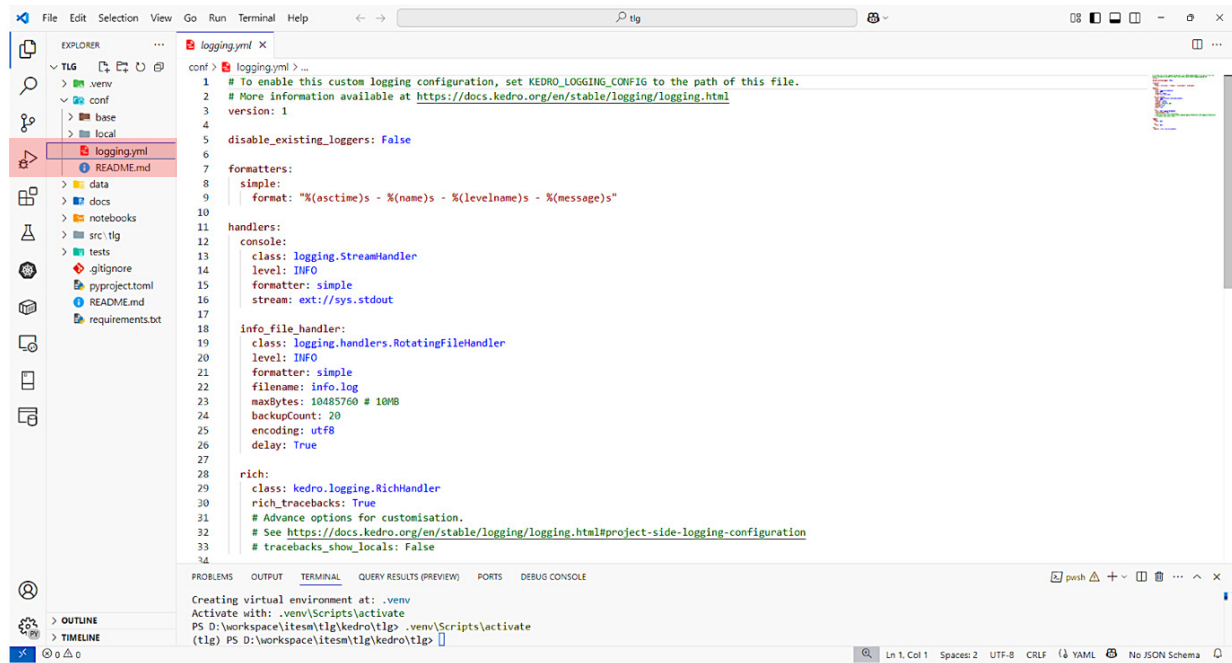


Figura 14. Configuración básica con un archivo YAML.

Para utilizar esta configuración, se debe definir una variable de ambiente de sistema operativo llamada `KEDRO_LOGGING_CONFIG`. Para Linux sería algo así:

```
export KEDRO_LOGGING_CONFIG=<project_root>/conf/logging.yml
```

La sintaxis dependerá del sistema operativo. Una vez definida la configuración, se puede utilizar la funcionalidad de bitácora en el código. Por ejemplo, para agregar bitácoras a un nodo, se puede utilizar el siguiente código:

```
# src/<package_name>/pipelines/<pipeline_name>/nodes.py
import logging

def process_data(data):
    """Procesa los datos de entrada"""
    logger = logging.getLogger(__name__)

    logger.info(f"Procesando dataset con {len(data)} registros")

    # Si encontramos valores nulos, registrarlo como advertencia
    null_count = data.isnull().sum().sum()
    if null_count > 0:
        logger.warning(f"Se encontraron {null_count} valores nulos en los datos")

    # Lógica de procesamiento...
    if some_error_condition:
        logger.error("Error en el procesamiento: datos inconsistentes")

    logger.info("Procesamiento completado exitosamente")
    return processed_data
```

Si se utiliza la configuración por *default*, todas las llamadas a `logger` enviará los registros de bitácora al archivo `info.log`.

Estrategias de *logging* para desarrollo en equipo

Establecer convenciones de equipo

- Qué niveles usar para diferentes tipos de eventos
- Qué información incluir en cada mensaje
- Qué no debe incluirse en *logs* (datos sensibles, etcétera.)

Creación de *loggers* específicos

Para facilitar el filtrado en las bitácoras, se pueden crear *loggers* específicos indicando el nombre durante la creación.

```
# Especificar el nombre completo del módulo
logger = logging.getLogger(__name__)

# Crear loggers con nombres específicos
data_quality_logger = logging.getLogger("data_quality")
model_performance_logger = logging.getLogger("model_
performance")
```

Incluir información de contexto

Esto facilita el análisis de las bitácoras.

```
def process_customer_data(data, customer_id):
    logger = logging.getLogger(__name__)

    # Incluir contexto en cada mensaje
    context = f"[Cliente: {customer_id}]"
    logger.info(f"{context} Iniciando procesamiento")

    # Procesamiento...

    logger.info(f"{context} Procesamiento completado")
```

Hooks para *logging* centralizado

Puedes utilizar *hooks* para implementar políticas de *logging* consistentes:

```
class LoggingStandardizationHooks:
    @hook_impl
    def before_node_run(self, node):
        logger = logging.getLogger(__name__)
        logger.info(
            f"Ejecutando nodo: {node.name} | "
            f"Entradas: {node.inputs} | "
            f"Salidas: {node.outputs}"
        )

    @hook_impl
    def after_node_run(self, node, inputs, outputs):
        logger = logging.getLogger(__name__)

        # Registrar estadísticas básicas de outputs para
        datasets tabulares
        for name, data in outputs.items():
            if hasattr(data, "shape"):
                logger.info(f"Dataset '{name}' shape: {data.
shape}")
```


Mejores prácticas de *logging* en Kedro

1. Mantener *logs* informativos pero concisos.

```
# ❌ Demasiado general
logger.info("Procesando datos")

# ✅ Claro y específico
logger.info(f"Procesando {len(data)} registros del dataset 'customer_transactions'")
```

Figura 15. Ejemplo de *logs* informativos

2. Usar niveles de *log* adecuadamente.

```
# Información para depuración
logger.debug(f"Valores de columnas detectados: {columns}")

# Eventos normales
logger.info("Modelo entrenado correctamente con R² de 0.87")

# Condiciones inusuales pero no errores
logger.warning("10% de los registros contienen valores faltantes")

# Errores recuperables
logger.error("No se pudo conectar a la base de datos, reintentando en 30s")

# Errores fatales
logger.critical("Corrupción detectada en el dataset principal, abortando ejecución")
```

Figura 16. Ejemplo de uso de niveles de *logs*.

3. Incluir información accionable.

```
# ❌ Sin detalles útiles
logger.error("Error al procesar datos")

# ✅ Con detalles para solucionar el problema
logger.error(
    "Error al procesar datos: columna 'customer_id' no encontrada. "
    "Verifique el catálogo y el esquema de datos de entrada."
)
```

Figura 17. Ejemplo de información accionable

4. Evitar información sensible.

```
# ❌ Incluye datos sensibles
logger.info(f"Procesando pago para tarjeta {credit_card_number}")

# ✅ Sin datos sensibles
logger.info(f"Procesando pago para tarjeta terminada en {credit_card_number[-4:]}")
```

Figura 18. Ejemplo de información sensible

5. Medir rendimiento con *logging*.

```
import time

def process_data(data):
    logger = logging.getLogger(__name__)
    start_time = time.time()

    # Procesamiento...

    execution_time = time.time() - start_time
    logger.info(f"Procesamiento completado en {execution_time:.2f} segundos")
```

Figura 19. Ejemplo de rendimiento con *logging*.

6. Utilizar *hooks* para depuración.

```
class DebuggingHooks:
    @hook_impl
    def before_node_run(self, node, inputs):
        """Imprimir un resumen de los inputs antes de ejecutar un nodo."""
        logger = logging.getLogger(__name__)

        for name, data in inputs.items():
            if hasattr(data, "shape"):
                # Para datos tabulares
                logger.debug(f"Input '{name}' shape: {data.shape}")
            if hasattr(data, "dtypes"):
                logger.debug(f"Input '{name}' dtypes: {data.dtypes}")
            elif isinstance(data, dict):
                # Para diccionarios
                logger.debug(f"Input '{name}' keys: {list(data.keys())}")
            else:
                # Para otros tipos
                logger.debug(f"Input '{name}' type: {type(data).__name__}")
```

Figura 20. Ejemplo de uso de *hooks* para depuración.

7. Adicionalmente, en producción se pueden utilizar identificadores de correlación (normalmente UUIDs) para rastrear flujos completos.

```
import uuid
from kedro.framework.hooks import hook_impl
from kedro.pipeline.pipeline import Pipeline

class CorrelationIDHooks:
    @hook_impl
    def before_pipeline_run(self, pipeline: Pipeline):
        """Añadir un ID de correlación al contexto de ejecución."""
        self.correlation_id = str(uuid.uuid4())
        logger = logging.getLogger(__name__)
        logger.info(f"Iniciando pipeline con ID de correlación: {self.correlation_id}")

    @hook_impl
    def before_node_run(self, node):
        """Incluir el ID de correlación en cada log de nodo."""
        # Añadir el ID de correlación al contexto de logging
        logging.LoggerAdapter(
            logging.getLogger(node.name),
            {"correlation_id": self.correlation_id}
        )
```

Figura 21. Ejemplo de identificadores de correlación.

Conclusión

Un sistema de *logging* bien configurado es una herramienta fundamental para el mantenimiento efectivo de proyectos Kedro. Proporciona visibilidad sobre:

1. **Progreso de ejecución:** Seguimiento paso a paso del *pipeline*.
2. **Problemas y errores:** Detección y diagnóstico rápido.
3. **Rendimiento:** Identificación de cuellos de botella.
4. **Auditoría:** Registro histórico de ejecuciones y resultados.
5. **Calidad de datos:** Monitoreo de anomalías y problemas.

Al implementar una estrategia coherente de *logging* en el proyecto, los equipos pueden:

- Acelerar el tiempo de depuración.
- Facilitar el monitoreo en producción.
- Mejorar la comunicación entre miembros del equipo.
- Garantizar la trazabilidad de las ejecuciones.
- Cumplir con requisitos de auditoría y cumplimiento.

Es importante recordar que el *logging* efectivo no se trata solo de registrar todo, sino de registrar la información correcta en el momento adecuado y en el formato más útil para cada caso de uso.

Integración con control de versiones (Git) y mejores prácticas

El control de versiones es esencial para el desarrollo de proyectos de datos, especialmente cuando se trabaja en equipo. Kedro está diseñado para integrarse perfectamente con sistemas de control de versiones, facilitando la colaboración y el seguimiento de cambios en el código.

Por defecto, Kedro considera **Git como el sistema de control de versiones a utilizar**; pero, siendo un proyecto de Kedro un proyecto de código de Python, se puede utilizar cualquier otro sistema, como SVN o Mercurial. En esta sección nos enfocaremos en Git.

Cuando se crea un nuevo proyecto con `kedro new`, Kedro genera automáticamente un archivo `.gitignore` apropiado. Este archivo incluye patrones para excluir:

- Archivos de configuración de IDE y editores.
- Archivos de caché de Python.
- Entornos virtuales.
- Archivos de credenciales y configuraciones locales.
- Datos grandes y archivos temporales.

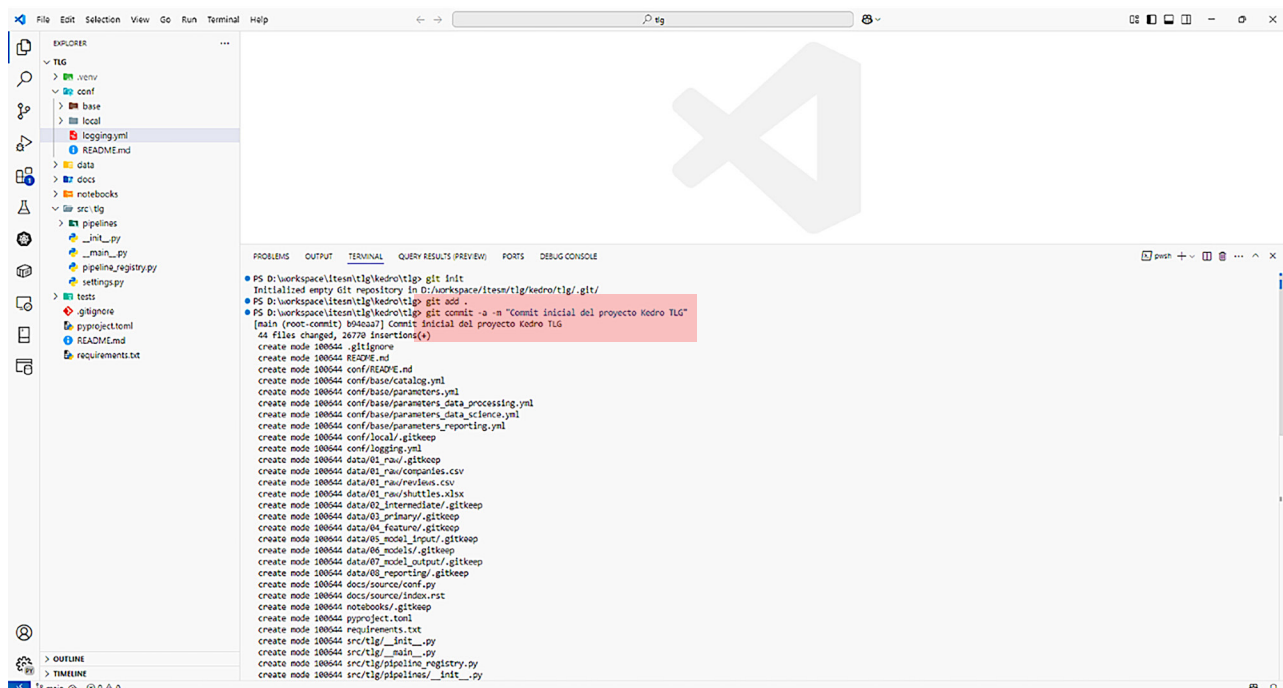
Es importante destacar que, por defecto, Kedro excluye:

- El directorio `/conf/local/`, que contiene configuraciones específicas de cada desarrollador.
- El directorio `/data/`, que suele contener archivos grandes que no son adecuados para Git.
- El directorio `/logs/`, que contiene archivos generados durante la ejecución.

Para empezar a trabajar con Git, se debe inicializar el proyecto ejecutando las siguientes instrucciones en la carpeta del proyecto, desde una terminal.

```
git init
git add .
git commit -m "Commit inicial del proyecto Kedro"
```

En la terminal de VSCode, sería algo como lo siguiente:



The screenshot shows the VSCode interface with a terminal window open. The terminal displays the following commands and output:

```
PS D:\workspace\litemit\kedro\litemit> git init
Initialized empty Git repository in D:/workspace/litemit/kedro/litemit/.git/
PS D:\workspace\litemit\kedro\litemit> git add .
PS D:\workspace\litemit\kedro\litemit> git commit -m "Commit inicial del proyecto Kedro TLG"
[master (root-commit) b040a2] Commit inicial del proyecto Kedro TLG
44 files changed, 28778 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 conf/README.md
create mode 100644 conf/base/catalog.yml
create mode 100644 conf/base/parameters.yml
create mode 100644 conf/base/parameters_data_processing.yml
create mode 100644 conf/base/parameters_data_science.yml
create mode 100644 conf/base/parameters_reporting.yml
create mode 100644 conf/local/.gitkeep
create mode 100644 conf/logging.yml
create mode 100644 data/Q1_raw/.gitkeep
create mode 100644 data/Q1_raw/companies.csv
create mode 100644 data/Q1_raw/reviews.csv
create mode 100644 data/Q1_raw/shuttles.xlsx
create mode 100644 data/Q2_intermediate/.gitkeep
create mode 100644 data/Q3_primary/.gitkeep
create mode 100644 data/Q4_features/.gitkeep
create mode 100644 data/Q5_model_input/.gitkeep
create mode 100644 data/Q6_model/.gitkeep
create mode 100644 data/Q7_model_output/.gitkeep
create mode 100644 data/Q8_reporting/.gitkeep
create mode 100644 docs/source/conf.py
create mode 100644 docs/source/index.rst
create mode 100644 notebooks/.gitkeep
create mode 100644 pyproject.toml
create mode 100644 requirements.txt
create mode 100644 src/tlg/_init_.py
create mode 100644 src/tlg/_main_.py
create mode 100644 src/tlg/pipeline_registry.py
create mode 100644 src/tlg/pipelines/_init_.py
```

Figura 22. Terminal VSCode.

Para sincronizar con un repositorio remoto, se debe configurar:

```
git remote add origin <url_del_repositorio>
git push -u origin main
```

Dependiendo del tipo de servicio, es posible que requieras introducir o configurar algún tipo de credenciales o certificados. Consulta la documentación de tu servicio para esto.

El uso de Git queda fuera del alcance de este curso, si deseas más información, podrás encontrar el enlace de la documentación oficial en la sección de Material adicional de esta lectura.

Además del control de versiones de código, que se maneja con Git o alguna otra herramienta, Kedro también proporciona versionado para los datos definidos en los catálogos.

Para esto, hay que agregar el parámetro **versioned: true** a la configuración del catálogo.

```
# conf/base/catalog.yml
model:
  type: pickle.PickleDataset
  filepath: data/06_models/model.pkl
  versioned: true # Habilita el versionado para este dataset
```


Cuando se habilita el versionado de datos, Kedro almacena cada versión en una subcarpeta con marca de tiempo, por ejemplo:

```
data/06_models/model.pkl/  
├── 2023-05-01T12.30.45.123Z/  
│   └── model.pkl  
├── 2023-05-02T15.45.30.456Z/  
│   └── model.pkl  
└── ...
```

Figura 23. Versionado de datos activado.

Por omisión, Kedro carga la última versión del conjunto de datos. Se puede cargar una versión específica al ejecutar el *pipeline* indicándola con el parámetro `--load-versions`, por ejemplo, para cargar la versión de la fecha 2025-05-01 12:02:03.125 del *dataset* `cars`, sería algo así:

```
kedro run --load-versions=cars:2025-05-01T12.02.03.125Z
```

No todos los tipos de *datasets* soportan versionamiento, solamente aquellos que heredan de la clase `AbstractVersionedDataset`. Esto se puede verificar directamente en el código fuente del *dataset*, y lo puedes consultar en el material adicional.

Por ejemplo, el *dataset* `pandas.CSVDataset` está definido como:

```
class CSVDataset(AbstractVersionedDataset[pd.DataFrame, pd.  
DataFrame]):
```

Lo que indica que sí se puede meter en esquema de versiones.

Colaboración en equipo con Kedro y mejores prácticas

En un equipo trabajando con Kedro, es común encontrar los siguientes roles:

- **Data Engineers:** responsables de *pipelines* de ingesta y procesamiento.
- **Data Scientists:** responsables de *pipelines* de modelado y experimentación.
- **MLOps Engineers:** responsables de despliegue y monitoreo.
- **Team Leader:** supervisa la arquitectura y mantiene la integridad del proyecto.

Para trabajar eficientemente en el mismo proyecto es necesario:

- **Dividir por *pipelines*:** asignar responsabilidad de *pipelines* específicos a diferentes miembros.
- **Tener reuniones de sincronización:** revisar cambios y planificar integraciones.
- **Documentación clara:** mantener documentación actualizada sobre la estructura del proyecto.
- **Convenciones compartidas:** establecer y seguir convenciones de nombrado y estilo.

Algunas de las mejores prácticas para equipos grandes o proyectos complejos son:

1. Modularizar más allá de los *pipelines*

```
src/  
├── <package_name>/  
│   ├── pipelines/  
│   │   ├── data_ingestion/  
│   │   ├── data_processing/  
│   │   └── data_science/  
│   ├── io/                # Datasets personalizados  
│   │   └── custom_datasets.py  
│   ├── utils/             # Utilidades compartidas  
│   │   ├── data_validation.py  
│   │   └── metrics.py  
│   └── hooks/              # Hooks organizados por función  
│       ├── logging_hooks.py  
│       └── performance_hooks.py
```

Figura 24. Ejemplo de estructura de módulos.

2. Documentación detallada

Mantener documentación completa para facilitar la incorporación de nuevos miembros:

- 🌟 **README.md principal:** visión general del proyecto y configuración.
- 🌟 **README.md por *pipeline*:** descripción detallada de cada *pipeline*.
- 🌟 **Diagrama de arquitectura:** visión general de los componentes del sistema.
- 🌟 **Registros de decisiones (ADRs):** documentar decisiones arquitectónicas importantes.

3. Automatización de tareas repetitivas

- Usar *hooks* o *scripts* para automatizar tareas comunes.

4. Estrategias de ramificación para experimentación

Cada rama de experimento puede tener su propio archivo de parámetros. Para equipos de ciencia de datos con múltiples experimentos:

```
main
├── develop
│   ├── feature/ingestion-system
│   ├── feature/processing-pipeline
│   └── experiment/models
│       ├── experiment/random-forest
│       ├── experiment/gradient-boosting
│       └── experiment/neural-network
```

Figura 25. Estructura con ramificación de experimentos.

5. Gestión de datos sensibles

Nunca almacenar credenciales en el repositorio:

- Usar `conf/local/credentials.yml` (ignorado por Git).
- Usar variables de entorno.
- Considerar gestores de secretos como Vault o AWS Secrets Manager.

Para pruebas con datos sensibles:

- Crear conjuntos de datos sintéticos o anonimizados.
- Mantener estos datos en `/data/test/` (versionado con Git).
- Agregar pruebas específicas que verifiquen el funcionamiento con estos datos.

Antes de terminar...

La integración efectiva de Kedro con Git proporciona numerosos beneficios, como la **trazabilidad**, que permite mantener un historial claro de cambios y la evolución del proyecto; la **colaboración**, al facilitar que múltiples personas trabajen de forma simultánea; la **experimentación segura**, ya que se pueden probar nuevos enfoques sin afectar el código principal; la **reversibilidad**, con la capacidad de volver a versiones anteriores cuando sea necesario; y finalmente, el **despliegue** continuo, que sienta las bases para implementar flujos de CI/CD.

Seguir estas mejores prácticas ayudará a los equipos a mantener proyectos organizados, colaborativos y listos para producción, incluso a medida que crecen en complejidad y tamaño.

Cierre

Los proyectos de datos modernos enfrentan una serie de desafíos críticos que la **organización eficiente** ayuda a resolver:

- **Complejidad creciente:** a medida que los proyectos de datos evolucionan, tienden a acumular más componentes, *pipelines* y dependencias. Una estructura modular bien definida mediante la separación en paquetes y módulos facilita gestionar esta complejidad de manera sostenible.
- **Colaboración efectiva:** el trabajo en equipo se convierte en un desafío cuando no existen convenciones claras sobre cómo estructurar y organizar el código. La adopción de prácticas estandarizadas y el uso de Git permiten que múltiples personas contribuyan simultáneamente sin obstaculizarse.
- **Reproducibilidad:** uno de los mayores desafíos en proyectos de datos es garantizar que los resultados sean reproducibles. La combinación de control de versiones para el código, versionado de datos y configuración sistemática de logs permite reconstruir exactamente cómo se generaron los resultados.

- **Mantenibilidad a largo plazo:** los proyectos de datos suelen tener ciclos de vida extensos, durante los cuales pueden cambiar los miembros del equipo. Una estructura organizada, documentación exhaustiva y convenciones consistentes facilitan que nuevos miembros comprendan y contribuyan al proyecto.

Implementar estas prácticas no solo mejora el desarrollo técnico, sino que aporta valor a nivel organizacional:

Valor organizacional	Descripción del valor
Reducción de deuda técnica	Previene la acumulación de soluciones <i>ad hoc</i> que generan problemas a largo plazo.
Aceleración del ciclo de desarrollo	Modularidad, pruebas automatizadas e integración continua permiten iterar más rápidamente.
Transferencia de conocimiento	Estructura estandarizada y documentación facilitan la incorporación de nuevos miembros.
Gobernanza y cumplimiento	<i>Logs</i> adecuados y control de versiones proporcionan trazabilidad necesaria.

Pregúntate: ¿cómo se gestionan estos proyectos actualmente en tu organización?

Material adicional

git (s.f.) *Book* [Libro]. git. Recuperado el 04 de septiembre de 2025, de: <https://git-scm.com/book/en/v2>

Kedro. (s.f.) *A toolbox for production-ready data science* [Una caja de herramientas para ciencia de datos lista para producción]. Kedro. Recuperado el 04 de septiembre de 2025, de: <https://docs.kedro.org/en/1.0.0/index.html>

Kedro. (s.f.) *Data versioning* [Control de versiones de conjuntos de datos]. Kedro. Recuperado el 04 de septiembre de 2025, de: https://docs.kedro.org/en/1.0.0/catalog-data/data_catalog/#dataset-versioning

Python. (s.f.) *logging — Logging facility for Python* [*logging* — Herramienta de registro para Python]. Python. Recuperado el 04 de septiembre de 2025, de: <https://docs.python.org/3/library/logging.html>

Repositorios:

https://github.com/kedro-org/kedro/blob/main/kedro/framework/project/default_logging.yml

https://github.com/kedro-org/kedro-plugins/tree/main/kedro-datasets/kedro_datasets.

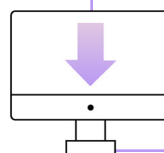
Nota: Los enlaces a las páginas web incluidos en este material adicional fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.

Bibliografía

Kedro. (s.f.) *Kedro project directory structure* [Estructura de directorios de un proyecto Kedro]. Kedro. Recuperado el 04 de septiembre de 2025, de: https://docs.kedro.org/en/1.0.0/getting-started/kedro_concepts/#kedro-project-directory-structure

Python package index (s.f.) *kedro 0.19.11*. Python package index. Recuperado el 04 de septiembre de 2025, de: <https://pypi.org/project/kedro/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.